



本科生课程报告

软件工程化说明文档

寻迹校园失物招领平台

课程名称： 软件工程

项目名称： 寻迹校园失物招领平台

团队名称： 寻迹项目组

项目负责人： 朱梓涵

团队成员： 朱梓涵、周星辰、林洪宽、郭焜华、程文韬

负责人学号： 24325356

文档版本： V1.0

基线日期： 2026 年 7 月 12 日

中山大学

2026 年 7 月

目录

1 文档目的与成果概览	1
2 开发过程与交付策略	1
3 需求、设计与接口契约	2
4 仓库组织与责任边界	4
5 Git、分支与评审协作	4
6 代码规范与静态约束	5
7 依赖、环境与可重复构建	6
8 数据库迁移与结构演进	7
9 测试分层与自动化质量	8
10 持续集成流水线	9
11 质量闭环与成果巩固	11
12 工程度量与可追溯交付	12
13 持续优化与扩展方向	12
14 结论	13

1 文档目的与成果概览

本文说明「寻迹」校园失物招领平台如何把需求、设计、编码、测试、配置、持续集成和质量管理组织为可重复的软件工程过程。本文是截至 2026 年 7 月 12 日的课程期末成果版，集中呈现需求与架构文档、源代码、依赖锁、Alembic 迁移、GitHub Actions、测试结果与 Git 协作记录共同构成的工程化交付。

表 1: 工程化成果概览

成果类别	已完成内容	主要交付
设计基线	需求、架构、API、枚举、数据库与测试口径统一管理	docs/ 系列文档
工程实现	模块化代码、锁文件、迁移、容器配置和持久任务已组织入库	Backend、AI、双前端与六服务 Compose
质量校验	Ruff、Mypy、Pytest、vue-tsc、Vitest 与生产构建纳入 CI	五个并行 Job 全部通过
协作交付	分支、提交、评审、行尾和忽略规则形成团队约定	CONTRIBUTING.md、 .gitattributes、期末报告

2 开发过程与交付策略

2.1 轻量迭代模型

项目采用「先闭环、再增强、后打磨」的轻量迭代，而不是重型阶段模型。需求先按 P0、P1、P2 分级：P0 保证登录、发布、搜索、匹配、认领、交接、通知和后台审核闭环；P1 增加综合评分、敏感保护、信用、举报、统计与批量录入；P2 收纳凭证审核、申诉、邮件通知、长期未领公告和更强模型。新需求若影响数据库、接口、页面或里程碑，则进入后续增量迭代，使期末交付始终聚焦已定义的课程主流程。

表 2: 迭代与可交付增量

阶段	目标	主要产物	工程验收点
A	需求冻结与选型	需求基线、分工、仓库结构、接口草案	P0/P1/P2 范围明确，角色和文档入口建立

阶段	目标	主要产物	工程验收点
B	概要与设计	模块、数据库、枚举、接口规范、页面清单	字段、状态和错误码形成书面契约
C	P0 功能骨架	双前端、主后端、AI 基线、迁移和基础联调	主路径可运行，关键 service 有自动化测试
D	P1 增强	匹配、积分、举报、统计、敏感策略	状态副作用、审计和降级路径补齐
E	集成与答辩	回归、部署、演示数据、报告和脚本	质量命令通过，报告、PDF 与验收记录归档

2.2 完成定义与例会机制

仓库定义的完成条件包括：代码进入对应功能分支、至少一次同伴 review、接口或字段变化同步文档、关键路径完成自测并通过课程主流程验收。周初确定目标、任务和分支；周中同步联调阻塞及接口/数据库变化；周末验收增量并更新文档与测试状态。联调问题应包含请求、响应和复现步骤，避免只保留在即时聊天中。

团队工作约定由 CONTRIBUTING.md、CI 和提交历史共同承载；提交前的静态检查、测试和构建已组成统一的课程交付流程。GitHub required checks、分支保护和评审策略可作为团队后续扩展的平台治理能力。

3 需求、设计与接口契约

3.1 文档即契约

项目将容易漂移的信息拆成权威文档，并在 CODEBUDDY.md 中按任务给出阅读路由。枚举值以 docs/architecture/enums.md 为唯一字典；JSON/数据库命名、时间、响应包、分页、ULID 和错误码以 docs/api/conventions.md 为准；匹配与信用规则分别由 matching-rules.md 和 credit-rules.md 管理；数据库字段由设计文档与 Alembic 共同演进。接口、数据库、枚举或工作流变化要求文档和代码同一 PR 更新。

表 3: 契约来源与实现映射

契约主题	权威来源	实现/验证位置
需求范围与优先级	docs/project-management/requirements-baseline.md	页面、router/service 和验收清单
角色、枚举与状态	docs/architecture/enums.md、 interaction-logic-model.md	Pydantic Literal、service 守卫、前端共享类型
API 命名与响应	docs/api/conventions.md、 模块 API 文档	backend/app/common/response.py、schemas、前端 HTTP 封装
数据结构与迁移	docs/architecture/database-design.md	SQLAlchemy models、 backend/alembic/versions/
匹配和降级	matching-rules.md、 docs/api/ai-service.md	backend AIClient、match service、AI baseline tests
信用与防重	credit-rules.md	CreditService、唯一约束、 service tests
测试口径	docs/testing/test-plan.md、 test-cases.md	pytest/Vitest 文件、CI 与 acceptance checklist

3.2 需求追踪链

interaction-logic-model.md 已给出 P0 需求到页面、API/service、状态和测试的追踪矩阵。工程化报告建议用如下最小链路维护每个变更：

需求编号 → 页面/用例 → API Schema → Service 状态迁移 → 迁移/数据约束 → 自动化测试 → 验收记录

例如 P0-08 交接确认对应用户端认领详情、确认 API、ClaimService、claim、found、lost、handover 状态变化、积分唯一键、通知与日志，以及双确认 service 测试。链路的价值在于修改任何一端时都能判断受影响的文档、表和测试，而不是只验证单个接口返回成功。

4 仓库组织与责任边界

仓库采用 monorepo，使跨端类型、接口文档、迁移和部署配置能在同一提交内评审：

表 4: 顶层目录与工程责任

目录	内容	责任边界
frontend/ user-app/	用户端 Vue SPA	移动优先交互、表单、路由、状态和 API 调用
frontend/ admin-web/	管理端 Vue SPA	审核、治理、统计和管理员会话
frontend/ shared/	TypeScript 枚举、类型、标签与选项	双前端共享的接口字面量和响应类型
backend/	FastAPI 模块化单体、Alembic 和 pytest	业务真相、状态、权限、事务和持久化
ai-service/	独立 FastAPI 计算服务与测试	分类、匹配、敏感检测及规则降级，不连接业务库
deploy/	Compose、环境模板与说明	可重复部署和本地联调
docs/	需求、架构、API、项目管理和测试	设计基线、决策与验收材料
report/	七项期末书面交付物与公共模板	从仓库事实生成可归档报告

WBS 将项目管理/QA/文档、用户前端、用户与物品后端、认领与治理后端、AI/运维分配给五类角色，并规定交叉 review 责任。个人贡献由 issue、PR、Git 提交、文档和团队确认综合记录，使代码、协作和书面交付都能在期末进行追溯。

5 Git、分支与评审协作

5.1 分支和提交约定

CONTRIBUTING.md 定义 main 为可发布分支、develop 为集成分支，功能和修复分别使用 feature/<name>、fix/<name>。提交采用短 Conventional Commit，相关时携带 FR 编号，例如：

```
feat(FR-02): add lost item publish api
```

```
fix(FR-05): correct claim review status transition
refactor(claim): extract handover transaction into service
test(match): cover score fallback path
docs: update database model
```

每个 PR 应只处理一个功能或缺陷，关联 issue/FR，说明验证记录；UI 变化附截图，API、枚举、数据库字段或流程变化同步对应文档。CI 对 main/develop 的 push 与 pull request 都运行，并使用 concurrency group 取消同一引用上的旧运行，减少过时结果占用资源。

5.2 跨平台文本与忽略规则

2026 年 7 月 12 日仓库新增 .gitattributes：所有文本自动识别并统一 LF，Windows *.bat 和 *.cmd 保持 CRLF，图片、PDF、压缩包和字体标记为 binary，避免行尾归一化破坏二进制文件。.gitignore 排除 .env、虚拟环境、node_modules、缓存、覆盖率与 LaTeX 中间产物，同时允许 .env.example 入库。该组合降低跨平台整文件 diff 和秘密误提交概率。Secret 扫描与凭据轮换属于生产化安全的持续扩展方向。

6 代码规范与静态约束

6.1 后端与 AI 服务

表 5: Python 工程规范

主题	约定	工具/实现
语言与类型	Python 3.12，公共函数/方法/返回值要求类型注解	Mypy strict + Pydantic plugin
格式与 lint	100 列，E/F/W/I/B/UP/SIM/N/ASYNC/RUF 规则	Ruff check 与 format check
异步边界	路由 async def，DB 用 AsyncSession，HTTP 用 httpx.AsyncClient	Ruff ASYNC、代码评审和测试

主题	约定	工具/实现
分层	router 只处理 HTTP/Schema/权限, service 处理业务, repository 处理 SQL	固定包结构、静态检查与代码评审
数据模型	SQLAlchemy ORM 与 Pydantic DTO 分离	models.py 与 schemas.py
ID 与命名	ULID 26 位; JSON camelCase、数据库 snake_case	ULID 工具、Pydantic alias、API 文档
事务与审计	多表写入同一会话提交, 关键状态写 operation log	行锁、CAS、唯一键、service tests

backend/pyproject.toml 和 ai-service/pyproject.toml 集中配置 Ruff、Mypy、Pytest 及 marker。测试允许 assert; FastAPI Depends 模式豁免 B008; 异步测试由 asyncio_mode=auto 驱动。后端覆盖率以 app/ 为源, 排除应用入口和迁移。

6.2 前端规范

双前端使用 Vue 3 Composition API、Pinia、Vue Router、Element Plus、TypeScript strict 和 SCSS。frontend/shared/src/ 汇总接口类型和枚举, Vite/tsconfig 以 @xunji/shared 指向同一源码, 减少双端重复定义。API 层统一 Axios 实例、15 秒超时、Bearer token、统一响应解包、业务错误对象、requestId 和未授权会话清理; router 负责页面访问体验, 后端仍是权限真相。

pnpm lint 通过 vue-tsc --noEmit 执行 TypeScript 类型校验, Vitest 覆盖共享工具、API 封装和关键交互逻辑。ESLint/Prettier、Vue 组件测试和浏览器端到端回归可在持续优化阶段扩展, 与现有类型、单元和构建校验形成更完整的前端质量链。

7 依赖、环境与可重复构建

7.1 依赖锁定

Python 模块使用 uv 管理 pyproject.toml+uv.lock, 前端分别提交 package.json+pnpm-lock.yaml。CI 使用 uv sync --all-extras --dev 和 pnpm install --frozen-lockfile; Docker 使用 uv sync --frozen --no-dev 与固定 pnpm 9.15.9。这样开发声明可以使用兼容范围, CI/镜像仍按锁文件解析到确定版本。

表 6: 模块依赖与构建契约

模块	运行时	锁文件	主要构建命令
Backend	Python $\geq$ 3.12	backend/uv.lock	uv sync ; uv run uvicorn app.main:app
AI Service	Python $\geq$ 3.12	ai-service/uv. lock	uv sync ; uv run uvicorn app.main:app
User App	Node 20、pnpm 9	frontend/ user-app/ pnpm-lock.yaml	pnpm build
Admin Web	Node 20、pnpm 9	frontend/ admin-web/ pnpm-lock.yaml	pnpm build
容器栈	Docker/Podman Compose	上述锁文件 + 镜像 tag	docker compose ... up --build

基础镜像和 MySQL/MinIO 使用明确 tag，结合四份依赖锁支持课程环境的可重复构建。镜像 digest、非 root 运行、SBOM 和镜像漏洞扫描纳入生产供应链的扩展方向。

7.2 配置管理

真实配置由环境变量和不入库的 .env 提供，仓库只保留带空秘密的示例。Compose 对 MySQL root、数据库、JWT、管理员、MinIO 等关键值使用必填展开；后端在非 DEBUG 环境拒绝默认 JWT/管理员密码，AI 服务要求共享环境的服务 token 至少 32 字符（显式本地模式除外）。本地 debug SMS 还需要 DEBUG、SMS_DEBUG_ENABLED 和精确手机号白名单同时满足，不能使用通配符。

8 数据库迁移与结构演进

数据库演进以 Alembic 为唯一真相源。backend/alembic/env.py 导入全部 ORM metadata，支持异步在线迁移和离线 SQL；开发使用 autogenerate 生成初稿，再由评审者核对数据迁移、索引、唯一约束、默认值和降级路径。常用命令为：

```
cd backend
uv run alembic upgrade head
uv run alembic revision --autogenerate -m "add xxx"
uv run alembic downgrade -1
uv run alembic heads
```

当前迁移从 0001/0002 基础表，经过内容审核字段、match pair 去重、治理审核意见、评分来源、举报唯一目标，到 0008 持久任务，形成单一线性 head。0004 在约束前重写外部引用，体现「先迁移数据、再收紧约束」；0007 完成重复举报整理与唯一约束，部署时按规范在数据库备份上核对历史记录。

历史 backend/sql/schema.sql 仅保留参考，不再作为部署入口。由旧脚本创建且没有 alembic_version 的数据库不能直接 stamp：必须先备份并逐项比较表、列、索引、唯一约束、默认值和 nullability，结构完全等价后才可一次性 stamp。CI 对迁移脚本的结构逻辑与单一 head 进行校验；MySQL 容器中的 upgrade/downgrade 属于持续集成扩展项。

9 测试分层与自动化质量

9.1 现有测试结构

表 7: 测试层次与课程成果

层次	工具/位置	已覆盖重点	课程用途
后端 service/接口	pytest、 pytest-asyncio、httpx ASGITransport	用户、物品、匹配、认 领、积分、通知、治理、 安全、任务	隔离环境验证业务 规则和状态迁移
AI 单元/路由	pytest、httpx mock transport	Schema、配置、服务 token、客户端异常、 规则降级、URL 安全	验证离线规则基线 和服务契约
前端工 具/API	Vitest	HTTP 错误、格式化、 导航、交互冲突、会话 辅助函数	验证双前端共享逻辑
静态检查	Ruff、Mypy、vue-tsc	Python 风格/异步/类 型，TypeScript strict	在提交前发现类型 与规范问题

层次	工具/位置	已覆盖重点	课程用途
构建检查	Vite build、 Docker build、 Compose config	生产前端产物、 Backend/AI 镜像、 Compose 展开	验证可重复构建与部署配置
验收走查	acceptance checklist	P0/P1 主流程与 service 路径	统一记录功能验收结果

后端测试 fixture 每个测试通过 SQLAlchemy metadata 在内存 SQLite 建表，并替换 FastAPI session 依赖；MinIO client 和签名器使用 mock，保持测试离线且可重复。该结构高效验证业务规则，MySQL 锁与真实 MinIO 权限校验可作为集成环境的持续扩展项。

9.2 最近一次记录结果

截至基线日期，最近一次远端 CI 运行的自动化结果如下：

表 8: 最近一次远端 CI 自动化结果

模块	检查	记录结果
Backend	Ruff、Mypy、Pytest、Alembic heads	Ruff 通过；80 个源文件类型通过；200 passed；80% 覆盖率
AI Service	Ruff、Mypy、Pytest	Ruff 通过；14 个源文件类型通过；60 passed
User App	Vitest、production build	17 passed；构建通过
Admin Web	Vitest、production build	15 passed；构建通过
Compose/镜像	配置展开、Backend/AI 镜像构建	全部通过

10 持续集成流水线

10.1 触发与并发控制

GitHub Actions 工作流 `.github/workflows/ci.yml` 在 main/develop 的 push 和 pull request 触发。concurrency key 由 workflow 与 ref 组成，cancel-in-progress=true，

同一分支的新提交会取消旧运行。五个 job 默认并行：Backend、AI Service、User App、Admin Web 和 Docker build smoke；任一 job 异常都会使本次工作流失败，共同构成课程交付的自动化质量门禁。

10.2 CI 图

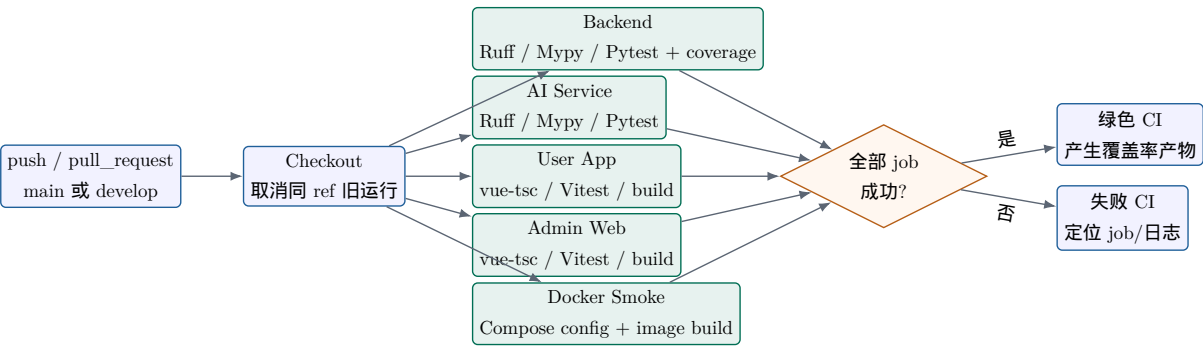


图 1: 寻迹持续集成流水线

10.3 CI 实际命令

表 9: CI job 与命令

Job	关键命令	产物/目的
Backend	<code>uv python install 3.12 ; uv sync --all-extras --dev ; ruff check . ; ruff format --check . ; mypy app ; pytest --cov=app --cov-report=xml --cov-report=term-missing</code>	上传 backend/coverage.xml , 即使测试失败也尝试保留
AI Service	<code>uv sync --all-extras --dev ; Ruff lint/format ; mypy app ; pytest</code>	验证服务离线基线、 Schema 和客户端路径
User App	<code>pnpm 9 + Node 20 ; pnpm install --frozen-lockfile ; pnpm lint ; pnpm test:unit --run ; pnpm build</code>	TypeScript、Vitest 和生 产包
Admin Web	与用户端相同的 frozen install、lint、 test 和 build	管理端类型、工具逻辑和 生产包

Job	关键命令	产物/目的
Docker Smoke	<code>docker compose ... config</code> <code>--quiet</code> ; 分别 <code>docker build</code> Backend/AI	配置可展开, 两个 Python 镜像可构建

Compose 配置检查注入非部署用的 CI 占位秘密, 以验证必填变量; Docker job 构建 Backend 与 AI 镜像, 双前端 job 同时完成生产包构建。六服务启动、健康等待与跨容器 API 测试作为生产化集成环境的后续扩展。

11 质量闭环与成果巩固

项目维护测试计划、验收清单和 P0/P1 修复记录。问题处理按「定位文件与行号、修改实现、增补回归、同步 API/架构文档、更新验收状态」的闭环执行。

表 10: 关键质量主题与实现措施

质量主题	已完成措施	持续优化方向
持久任务	<code>durable_jobs</code> 、租约、退避、回收与版本去重	MySQL 多 worker 压力与故障演练
敏感图片	<code>bucket</code> 私有、稳定引用、短签名、权限后签名、fail-closed	MinIO 角色化端到端验收
匹配可解释性	<code>imageAvailable/degraded/source</code> 与主后端总分重算	真实图像模型选型和效果评估
并发状态	行锁、CAS、唯一约束和统一冲突错误	MySQL 并发认领、审批与双确认压测
配置安全	非 DEBUG 拒绝默认值, 短信调试码限定精确白名单	外部 <code>secret manager</code> 、凭据轮换与集中限流
交付一致性	需求、代码、测试和七份期末报告同步归档	从 OpenAPI/权威枚举生成共享契约

问题采用 P0、P1、P2、P3 优先级管理, 课程主流程在期末版中完成回归, 安全、并发、性能、任务观测与可访问性等生产化主题纳入持续优化 backlog。

12 工程度量与可追溯交付

项目稳定记录各模块测试通过数、Python 覆盖率 XML、Mypy 检查源文件数、迁移 head、前端构建结果、CI job 状态和 Git 提交/作者。这些指标用于复核可重复构建、主流程回归和变更追溯性。

表 11:可追溯交付记录

交付对象	归档内容	协作要求
功能 PR	issue/FR、变更摘要、测试命令、UI 截图或 API 样例	保持单一主题和清晰评审边界
API 变更	对应 API 文档、Schema、共享 TS 类型和契约测试	枚举和错误码同步更新
数据库变更	Alembic upgrade/downgrade、数据库设计和备份/迁移说明	必须核对数据迁移，不只看 autogenerate
问题修复	复现步骤、回归测试、修复提交和验证结果	同步评估相关数据与配置
发布/演示版本	commit/tag、锁文件、迁移 head、环境模板和验收记录	冻结代码和演示数据
报告	基线日期、实现状态、结果表	图表与当前实现保持一致与可追溯路径

持续优化可扩展覆盖率阈值、问题关闭周期、构建时长趋势、durable job 积压/延迟、AI 降级率与 P0 浏览器通过率，使工程度量直接服务发布与容量决策。

13 持续优化与扩展方向

13.1 集成环境扩展

可在 MySQL 8.4 与 MinIO 环境中扩展 Alembic 升降级、并发认领/审批/双确认、匿名访问拒绝和角色签名链路测试。浏览器层可使用 Playwright 编排两个普通用户与一个管理员的发布、匹配、认领、申诉、交接和举报旅程，并将日志与截图归档到 CI。

13.2 代码与契约自动化

前端可增加 ESLint/Prettier、Vue 组件测试和覆盖率；仓库可增加 AST 架构依赖规则，并从 OpenAPI/权威枚举生成或校验 TypeScript 类型。事务处理可进一步统一模板、全局锁顺序和 MySQL deadlock 的整事务受限重试策略。

13.3 供应链与运行质量

生产化阶段可锁定基础镜像 digest，使用非 root、只读根文件系统和最小 capability；接入 dependency review、secret scan、SBOM、许可证与镜像扫描。运行层可增加 request ID 跨服务传播、结构化脱敏日志、任务指标、AI 降级率、告警和备份恢复演练。

表 12: 扩展后的发布门禁		
门禁层	必须通过	失败处理
快速 PR	Ruff/Mypy/Pytest、vue-tsc/Vitest/build、架构依赖与契约检查	禁止合并，定位到模块负责人
数据变更	Alembic 单 head、upgrade/downgrade、MySQL 数据迁移测试、备份说明	要求 DBA/后端交叉 review
集成夜跑	完整容器、MinIO 权限、并发状态、Playwright P0	形成阻塞发布的缺陷记录
发布候选	安全扫描、镜像、迁移、备份恢复、演示/生产 smoke	冻结版本，不允许临时绕过
发布后	健康、错误率、任务积压、AI 降级、关键旅程	回滚或按预案降级

14 结论

寻迹已经形成与课程规模匹配的工程基础：monorepo 统一交付、需求与枚举文档化、Python strict 类型和 Ruff、双前端 TypeScript strict、四份依赖锁、线性 Alembic、五 job CI、分层测试与质量闭环，以及跨平台行尾和环境忽略规则。最近远端 CI 中 Backend 200 项、AI Service 60 项、用户端 17 项和管理端 15 项测试全部通过，双前端构建与 Backend/AI 镜像构建同步通过。

至此，课程要求的「可运行、可测试、可部署、可追溯」工程化目标已形成闭环。持

续优化可进一步扩展真实依赖集成、浏览器 E2E、自动契约、供应链加固与运行观测，使当前课程成果可平滑演进为更高规模的工程交付体系。